FYP

Related Work:

This section talks about what other researchers have done before and how the current work (OhmNet) is different or better.

1. Learning Features from Networks

To understand how things (like proteins or cells) work, scientists often represent them as networks (like friendship networks, but for biological stuff). To make predictions, they need to extract useful "features" (key characteristics) from these networks. There are two main ways to do this:

Matrix Factorization: Breaks down the network into simpler mathematical parts.
But it has **problems**:
Doesn't handle networks that are very sparse (many missing connections).
Treats the whole network the same way, while sometimes local connections matter more.

Neural Network Embeddings (like Node2vec): Instead of math tricks, these use AI to learn features by:
Simulating "random walks" (like a tourist randomly walking through a city to explore neighborhoods).
Making sure nearby nodes in the network have similar features.

Balancing between finding similar nodes (homophily) and nodes with similar roles (structural equivalence).

Problem: These methods work well for single-layer networks (like one big friend group) but struggle with multi-layer networks (like different friend groups for work, family, hobbies).

If you ignore layers or merge them, you lose important details.
Some nodes might seem more important than they really are.

2. Biological Networks & Graphlets

In biology, scientists also predict protein functions by looking at small network patterns called graphlets (like counting how many triangles or squares a node is part of).

Graphlet methods give exact counts (discrete numbers).
Neural embeddings (OhmNet) give smooth, learned features (continuous numbers).
Problem: Graphlet methods can't easily handle multi-layer networks without simplifying them.

3. Deep Learning for Graphs
Some newer methods use deep learning (fancy AI) to learn features directly for a specific task (like predicting cancer genes).
These are task-dependent (only good for one job).
They don't handle multi-layer or hierarchical networks well.

Feature Learning in Multi-Layer Networks

This section explains how OhmNet learns useful features (key characteristics) of nodes (like proteins) in multi-layer networks (where each layer represents a different biological context, like different tissues).

1. The Problem
We have nodes (e.g., proteins) connected in different ways across multiple layers (e.g., protein interactions in brain, liver, heart, etc.).
Each layer is like a separate network, but they are related (e.g., brain layers are closer to each other than to liver layers).
Goal: Learn features (mathematical representations) for each node in each layer, so that:
Nodes with similar connections (network neighbors) have similar features.
Nodes in related layers (e.g., brain layers) have similar features.

2. How OhmNet Works
OhmNet has two main steps:

Step 1: Single-Layer Learning (Neighborhood Similarity)
For each layer, OhmNet learns features by:
Taking a node (e.g., Protein A in the brain layer).
Looking at its network neighbors (other proteins it interacts with).
Adjusting its features so that it can predict its neighbors well.
This is done using random walks (like exploring a maze randomly to find nearby nodes).
Key Idea: If two proteins have similar interaction patterns, their features should be similar.

Step 2: Hierarchical Learning (Layer Similarity)
Layers are organized in a hierarchy (like a family tree). Example:

Brain
├── Cortex
└── Brainstem
Liver

Nodes in related layers (e.g., cortex & brainstem) should have similar features.
OhmNet does this by:
Making sure a node's features in a child layer (e.g., cortex) are close to its features in the parent layer (e.g., brain).
This way, knowledge is shared across layers (**e.g., a protein in cortex is influenced by its role in the whole brain**).

3. Why This is Better
Old Methods:
Ignored layers (treated all connections as one big network).
Or treated layers independently (no sharing of information).
OhmNet's Advantages:

Efficient: Uses hierarchy to share information smartly (faster than comparing all layers).
Flexible: Learns features at multiple scales (e.g., brain-wide vs. cortex-specific).
Biologically Meaningful: Respects real-world relationships (e.g., brain tissues are more similar to each other than to liver).

4. Summary
OhmNet learns better features for nodes in multi-layer networks by:
Looking at local connections (neighborhoods in each layer).
Using a hierarchy to share knowledge between related layers.
This helps make more accurate predictions (like figuring out protein functions in different tissues)

3.3 Full OhmNet Model (Easy Explanation + Formulas)
Goal of OhmNet:
Learn feature representations (mathematical descriptions) for nodes (e.g., proteins) in a multi-layer network, where:

Each layer (e.g., brain, liver, heart) has its own network.

Layers are connected in a hierarchy (e.g., brain tissues are closer to each other than to the liver).

How OhmNet Does It:
OhmNet combines two key ideas:
1.Single-layer learning (neighborhood similarity).
2.Hierarchical learning (layer similarity).

Mathematical Formulation:
OhmNet solves this optimization problem:

What does this mean?

First term ($\sum \Omega_i$): Maximizes how well features predict neighboring nodes in each layer.
Ω_i = "How good are features at describing node connections in layer i?"
Second term ($\lambda \sum C$□): Encourages similar features in related layers**.**
C□ = "How different are features between a layer and its parent in the hierarchy?"
λ = Controls how much we care about layer similarity vs. single-layer accuracy.

Key Properties:
Learns features for each layer (f_1, f_2, ..., f_K).
Also learns features for internal hierarchy levels (f_{K+1}, ..., f_{|M|}).
Uses stochastic gradient descent (SGD) for efficient training.

3.4 The OhmNet Algorithm (Step-by-Step)
Phase 1: Neighborhood Sampling (Random Walks)

1. For each layer (e.g., brain network):
2. For each node (e.g., Protein A):
   Simulate random walks (like exploring a maze randomly).
   Record which nodes appear together (neighborhoods).

Phase 2: Feature Learning (Iterative Updates)
Initialize random features for all nodes in all layers.
Repeat until convergence:
For each layer (leaf in hierarchy):

1. Update features (f_i) to better predict its node neighborhoods (using SGD).

For each internal hierarchy level (e.g., "brain" parent node):

1. Update features (f□) to be closer to its children's features (e.g., cortex + brainstem).

Why It's Efficient:
Parallelizable: Different layers/hierarchy levels can be updated at the same time.
Hierarchy speeds things up:
Without hierarchy: Need $O(K^2N)$ comparisons (all layers vs. all layers).
With hierarchy: Only $O(|M|N)$ comparisons (parent-child relationships).

Summary (Key Takeaways)
OhmNet learns node features by:
Looking at local connections (random walks in each layer).
Using a hierarchy to share knowledge between related layers.
Balancing single-layer accuracy (Ω_i) and layer similarity (C□) with λ.
This makes it faster and more accurate than methods that ignore layer relationship.

Understanding Tissue-Specific Protein Networks (Simple Explanation)

What's This About?

Scientists built a **multi-layer protein network** showing:

1. **How proteins interact** in different body tissues (like brain vs. liver).
2. **How tissues are related** (e.g., retina → eye → nervous system).
3. **What jobs proteins do** in each tissue (e.g., cholesterol processing in blood).

---

1. The Tissue Family Tree (Hierarchy)

- **Source:** BRENDA Tissue Ontology (a biology "tissue dictionary").
- **Structure:**
    - **219 tissues** organized like a family tree (Fig. 3).
    - **107 blue leaves** = specific tissues (e.g., pancreas, retina).
    - **Parent nodes** = broader groups (e.g., "Eye" contains "Retina" and "Lens").
- **Relationships:** Defined by:
    - *"part_of"* (e.g., retina is part of the eye).
    - *"develops_from"* (e.g., sperm cells develop from reproductive tissues).

Why It Matters:
This helps OhmNet understand that proteins in related tissues (e.g., brain regions) should behave similarly.

---

2. Protein Networks for Each Tissue

- **Data Sources:** Trusted databases of protein interactions (HPRD, BioGRID, etc.).
- **Global Network:**
  - **21,557 proteins** connected by **342,353 interactions** (like a giant map of all protein handshakes).
- **Tissue-Specific Networks (107 layers):**
  - For each tissue (e.g., liver), they kept only interactions where:
    1. **Both proteins work in that tissue**, OR
    2. **One protein is tissue-specific + the other is found everywhere**.
  - Example: The *liver network* only shows proteins active in the liver.

Why It Matters:
Proteins do different jobs in different tissues—this captures those differences.

---

3. Protein Jobs in Each Tissue (Gene-Function Links)

- **Source:** Gene Ontology (GO) + manual work by Greene et al. (2015).
- **Process:**
  - Mapped **biological processes** (e.g., "oxygen transport") to tissues where they happen.
  - Only included functions with **≥15 known proteins** (to avoid guesswork).
- **Final Data:**
  - **584 tissue-specific functions** across **48 tissues**.
  - Example: *"Cholesterol processing"* is tagged to **blood plasma**.

Why It Matters:
This lets OhmNet predict what proteins *do* in specific tissues (e.g., "Does this protein help the brain or the liver?").

How OhmNet Works (Algorithm 1)

Goal: Learn **protein features** (mathematical profiles) for all tissues.

Step 1: Explore Protein Neighborhoods

- For each tissue (e.g., brain):
    - Simulate **random walks** (like exploring a maze) to find which proteins interact.
    - Record these groups (`Ni(u)` = protein `u`'s neighborhood).

Step 2: Learn Features

- **For specific tissues (blue leaves):**
    - Adjust features (`fi(u)`) to predict local interactions (using SGD).
- **For parent tissues (e.g., "NervousSystem"):**
    - Set features as the **average of child tissues** (e.g., brain + spinal cord features).
- Repeat until features stop improving.

Output:

- A **feature set** for every protein in every tissue, respecting both:
    1. **Local interactions** (protein neighbors in one tissue).
    2. **Global relationships** (similarities between tissues).

Why This is Cool

- **Old Methods:** Ignored tissue differences or treated them separately.
- **OhmNet's Power:**
    - Combines **tissue-specific data** + **hierarchy** for accurate predictions.
    - Example: Can predict if a protein causes **brain-specific diseases** (like Alzheimer's) but not liver problems.

Real-World Impact:
Helps design **precision medicine**—treatments tailored to specific tissues!

(Think of OhmNet as a "Google Maps" for proteins: it knows which roads (interactions) exist in which cities (tissues), and how cities are connected!)

5. Results (Simple Explanation)

*This section shows how **OhmNet** performs in real-world tasks compared to other methods.*

5.1 Predicting Protein Functions in Specific Tissues

Goal:

*Guess what **biological jobs** (functions) proteins do in different tissues (e.g., "Does this protein help the liver process cholesterol?").*

How They Tested:

1. ***Training:***
   - *OhmNet learns **unsupervised features** for proteins in all 107 tissues.*
   - *For each tissue + function (e.g., "blood plasma → cholesterol processing"), they trained a **classifier** using 90% of known protein-function pairs.*
2. ***Testing:***
   - *Predict functions for the **remaining 10% of proteins**.*

Competitors:

OhmNet was compared to:

- ***Other feature-learning methods** (e.g., Node2vec, LINE).*
- ***Function-prediction tools** (e.g., GeneMania, tissue-specific SVM).*

Results:

- ***OhmNet won by 10–26%** in accuracy (AUROC/AUPRC scores).*
- ***Why?** Other methods either:*
  - *Ignored tissue relationships (independent layers).*
  - *Merged all tissues into one network (collapsed layers), losing key details.*

Key Insight:

*Tissues are **hierarchically related** (e.g., brain regions are similar). OhmNet uses this to make better predictions!*

5.2 Transfer Learning: Predicting Functions in New Tissues

Goal:

*Guess protein functions in a **new tissue** (e.g., "brain") using data from **other tissues** (e.g., liver, heart).*

How They Tested:

1. ***Training:***
   - *Train classifiers on **all tissues except one** (the "target").*

2. ***Testing:***
    - *Predict functions for the **unseen target tissue**.*

Results:

- *OhmNet's predictions were **only 7% worse** than training on the target tissue itself!*
- *Worked best when **source and target tissues were related** (e.g., forebrain ↔ cerebellum).*

Why It Matters:

*This mimics real life—we often lack data for rare tissues/diseases. OhmNet can **borrow knowledge** from similar tissues!*

5.3 Visualizing Brain Protein Networks

Goal:

*Show that OhmNet's features **respect tissue hierarchy** (e.g., brain regions cluster together).*

Experiment:

1. *Built a **9-layer brain network** (e.g., cerebellum, brainstem).*
2. *Used OhmNet to **learn protein features**.*
3. *Plotted proteins in 2D space (like Google Maps for proteins).*

Results:

- ***Proteins from the same tissue clustered together** (e.g., all "midbrain" proteins grouped neatly).*
- ***Parent tissues (e.g., "brain") covered their children** (e.g., "brainstem" regions).*
- *Other methods (PCA/NMF) failed to show clear patterns.*

Cool Fact:

*OhmNet's **unsupervised** features automatically matched known biology—no manual tuning needed!*

Why OhmNet Rocks

1. ***Flexible:** Works for many tasks (function prediction, transfer learning).*
2. ***Accurate:** Beats rivals by **10–26%** by using tissue relationships.*
3. ***Interpretable:** Features match real-world biology (e.g., brain hierarchy).*

Real-World Use:

- *Design **tissue-specific drugs** (e.g., target Alzheimer's in the brain, not the liver).*
- *Study **rare diseases** with limited data (via transfer learning).*

(Think of OhmNet as a "polyglot scientist" that speaks the language of every tissue!)